GIT COMMANDS · STUDY NOTES

Git 基础命令总结

廖雪峰 Git 教程 · 五章基础命令精编

版本库 | 时光机穿梭 | 远程仓库 | 分支管理 | 标签管理

内容来源

廖雪峰官方网站 • Git 教程

抓取范围

5 章 21 页正文

命令条目

51 条（含速查表 49 条）

生成日期

2026 年 11 月 11 日

语法 命令的原始写法，占位符用尖括号标出

作用 一句话说清它解决什么问题、用在哪个环节

示例 教程原文的终端片段，可直接对照执行

注意 易错点与坑位提示：参数、顺序、误删风险

本手册由教程原文逐页抓取整理：每条命令给出语法、作用、典型示例与易错点，按章节与使用场景分组，便于随查随用。

目录 CONTENTS

01 **创建版本库** 5 条命令 3

1.1 准备目录（辅助命令） · 1.2 初始化仓库 · 1.3 添加与提交

02 **时光机穿梭** 13 条命令 5

2.1 查看状态与差异 · 2.2 查看提交历史 · 2.3 版本回退 · 2.4 撤销修改 · 2.5 删除文件

03 **远程仓库** 8 条命令 9

3.1 配置 SSH Key · 3.2 关联远程库 · 3.3 推送到远程 · 3.4 克隆与拉取

04 **分支管理** 17 条命令 12

4.1 查看、创建与删除分支 · 4.2 切换分支 · 4.3 合并分支 · 4.4 储藏工作现场（stash） · 4.5 多人协作 · 4.6 整理历史

05 **标签管理** 8 条命令 17

5.1 创建与查看标签 · 5.2 删除与推送标签

06 **常用命令速查表** 49 条 20

命令 → 作用 一览，按操作顺序排列，便于随查随用

建议阅读路径

01 **建仓与提交** `git init → git add → git commit` 第一章

02 **本地历史** `git status / diff / log → reset / reflog →` 撤销与删除 第二章

03 **协作与发布** `remote / push → branch / merge → tag` 第三 ~ 五章

阅读约定：语法中的 <xxx> 为占位符，尖括号不要照抄；以 $ 开头的是终端提示符，输入时不敲 $。

“注意”为易错点与坑位提示；示例优先采用教程原文中的真实输出，commit id 等因机器而异，以本机输出为准。

01

第一章

创建版本库

> 版本库（Repository）是一个被 Git 管理的目录，里面的每次修改、删除都能被追踪和还原。本章只做三件事：建目录、初始化仓库、把文件提交进去。

1.1 准备目录（辅助命令）

`pwd`	显示当前所在目录的绝对路径
`mkdir <dir>`	新建一个目录
`cd <dir>`	进入某个目录
`ls / ls -ah`	列出目录内容；-ah 可看到以点开头的隐藏文件
`vi <file>`	用 vi 编辑文件，输入 i 进入编辑，Esc 后 :wq 保存退出

#01 mkdir learngit / cd learngit / pwd

作用 在合适的位置建一个空目录并进入，再用 pwd 确认路径，这是建仓库前的准备动作。

```
$ mkdir learngit
$ cd learngit
$ pwd
/Users/michael/learngit
```

注意 Windows 下请确保目录名（包括父目录）不含中文，否则容易出现各种莫名其妙的问题。

1.2 初始化仓库

#02 git init

作用 把当前目录变成 Git 可以管理的仓库，并在目录下生成隐藏的 .git 目录。

```
$ git init
Initialized empty Git repository in /Users/michael/learngit/.git/
```

注意 .git 目录是 Git 跟踪管理版本库的核心，千万不要手动修改里面的文件，改乱了仓库就被破坏。若看不到它，用 ls -ah 查看（默认隐藏）。git init 必须在想变成仓库的那个目录下执行，已经存在的目录也能就地初始化。

1.3 添加与提交

#03 **git add <file>**

作用 把文件的修改添加到暂存区（stage），是提交前的第一步。

```
$ git add readme.txt
```

注意 执行后没有任何输出就是成功（Unix 哲学：没有消息就是好消息）。可反复多次执行以添加多个文件，如 git add file2.txt file3.txt。报 fatal: pathspec 'readme.txt' did not match any files 说明文件名写错或文件不在当前目录。

#04 **git add .**

作用 一次性把当前目录下所有新增和修改过的文件加入暂存区，省去逐个文件 add。

```
$ git add .
$ git status
```

注意 点号代表当前目录，会递归包含子目录。注意它会把当前目录下所有未忽略的改动都加进来，包括误建的临时文件；提交前务必用 git status 复核。更好的做法是先写好 .gitignore。

#05 **git commit -m "说明"**

作用 把暂存区里的所有修改一次性提交到当前分支，形成一条永久的历史记录。

```
$ git commit -m "wrote a readme file"
[master (root-commit) eaadf4e] wrote a readme file
 1 file changed, 2 insertions(+)
 create mode 100644 readme.txt
```

注意 -m 后面是本次提交说明，必须写得有意义。输出中 1 file changed 表示改动了 1 个文件，2 insertions(+) 表示插入了 2 行。强烈不建议省略 -m，说明对自己和别人阅读都很重要。

02

第二章

时光机穿梭

工作区、暂存区、版本库三层结构是 Git 的一切操作的基础：add 把修改放进暂存区，commit 把暂存区写进版本库。本章覆盖状态查看、历史查看、版本回退、撤销修改与删除文件。

2.1 查看状态与差异

#01 git status

作用 随时掌握仓库当前状态：哪些文件被修改、哪些已暂存、哪些还未被跟踪。

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)

        modified:   readme.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

注意 输出里 Untracked files 表示从未被添加过的新文件；nothing to commit, working tree clean 表示工作区干净、无待提交内容。

#02 git diff

作用 查看工作区与暂存区之间的具体差异，即“这次到底改了哪几行”。

```
$ git diff readme.txt
diff --git a/readme.txt b/readme.txt
index 46d49bf..9247db6 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,2 +1,2 @@
-Git is a version control system.
+Git is a distributed version control system.
 Git is free software.
```

注意 输出是 Unix 通用 diff 格式：- 开头是删除的行，+ 开头是新增的行。注意 git diff 只看“未暂存”的改动，已经 git add 的内容不会再显示。

#03 **git diff HEAD -- <file>**

作用 查看工作区与版本库中最新版本（HEAD）的差异，也就是“相比上次提交，我究竟改了什么”。

```
$ git diff HEAD -- readme.txt
diff --git a/readme.txt b/readme.txt
--- a/readme.txt
+++ b/readme.txt
@@ -1,4 +1,4 @@
-Git tracks changes.
+Git tracks changes of files.
```

注意 -- 后面跟文件名，用于把文件和分支名区分开。它把已暂存和未暂存的改动合并起来与 HEAD 比较，是确认“提交前到底还剩什么没提交”的利器。

2.2 查看提交历史

#04 **git log**

作用 按从近到远的顺序显示全部提交日志，含 commit id、作者、日期和提交说明。

```
$ git log
commit 1094adb7b9b3807259d8cb349e7df1d4d6477073 (HEAD -> master)
Author: Michael Liao <askxuefeng@gmail.com>
Date:   Fri May 18 21:06:15 2018 +0800

    append GPL
```

注意 commit id 是一串 SHA1 十六进制数，每个人、每次提交都不一样，以你自己的输出为准。回退之后，被“抛弃”的提交在 git log 里就看不到了，需要 git reflog 才找得回。

#05 **git log --pretty=oneline**

作用 每个提交只占一行，只保留 commit id 和说明，历史一多也不至于眼花。

```
$ git log --pretty=oneline
1094adb7b9b3807259d8cb349e7df1d4d6477073 (HEAD -> master) append GPL
e475afc93c209a690c39c13a46716e8fa000c366 add distributed
eaadf4e385e865d25c48e7ca9c8395c3f7dfaef0 wrote a readme file
```

注意 括号里的 HEAD -> master 说明当前 HEAD 指向 master 分支的这个提交。配合 --abbrev-commit 可只显示短 id。

2.3 版本回退

#06 git reset --hard HEAD^

作用 把当前分支回退到上一个版本，同时把工作区文件一并还原成那个版本的内容。

```
$ git reset --hard HEAD^
HEAD is now at e475afc add distributed
```

注意 HEAD 表示当前版本，HEAD^ 是上一个，HEAD^^ 是上上一个。--hard 会丢弃工作区与暂存区的改动，回退前务必确认。Windows 命令行下 ^ 可能被当作转义符，必要时写成 "HEAD^"。

#07 git reset --hard HEAD~n

作用 一次回退 n 个版本，n 为正整数，免去数一串 ^ 的麻烦。

```
$ git reset --hard HEAD~3
HEAD is now at eaadf4e wrote a readme file
```

注意 HEAD~1 等价于 HEAD^，HEAD~100 就是往上 100 个版本。数字写错会退到意料之外的版本，可用 git reflog 补救。

#08 git reset --hard <commit_id>

作用 把版本精确切换到指定的 commit，既能回退也能“回到未来”。

```
$ git reset --hard 1094a
HEAD is now at 83b0afe append GPL
```

注意 版本号不必写全，写前几位 Git 会自动去找，但不能只写一两位，否则可能匹配到多个版本而无法确定。三种模式：--hard 连工作区一起重置（改动全丢），--soft 只移动 HEAD（改动仍在暂存区），--mixed（默认）重置暂存区（改动退回工作区）。

#09 git reflog

作用 记录你执行的每一次命令（含被 reset 掉的提交），是回退之后的“后悔药”。

```
$ git reflog
e475afc HEAD@{1}: reset: moving to HEAD^
1094adb (HEAD -> master) HEAD@{2}: commit: append GPL
e475afc HEAD@{3}: commit: add distributed
```

注意 git log 只显示当前分支的历史，git reflog 才记录 HEAD 的移动轨迹。从输出里找到目标 commit id，再 git reset --hard 即可返回。

2.4 撤销修改

#10 git checkout -- <file> （等价 git restore <file>）

作用 丢弃工作区中该文件的修改，让它回到最近一次 git commit 或 git add 时的状态。

```
$ git checkout -- readme.txt
$ cat readme.txt
Git is a distributed version control system.
```

注意 命令中的 -- 非常重要，没有它就成了“切换到另一个分支”。此命令会用版本库里的版本覆盖工作区，未提交的改动将不可找回。新版 Git 推荐语义更清晰的 git restore <file>。

#11 git reset HEAD <file>

作用 把已 git add 到暂存区的修改撤出（unstage），重新放回工作区而不丢失内容。

```
$ git reset HEAD readme.txt
Unstaged changes after reset:
M       readme.txt
```

注意 撤销分两步：先 git reset HEAD <file> 退回工作区，再 git checkout -- <file> 丢弃改动。若已经 commit，就只能靠版本回退，且前提是还没推送到远程。

2.5 删除文件

#12 git rm <file>

作用 从工作区删除文件并把这次删除记入暂存区，随后 commit 即从版本库中删除。

```
$ git rm test.txt
rm 'test.txt'
$ git commit -m "remove test.txt"
[master d46f35e] remove test.txt
 1 file changed, 1 deletion(-)
 delete mode 100644 test.txt
```

注意 先手动删文件再 git rm 效果相同（等价于 rm 加 git add）。删错了可用 git checkout -- test.txt 从版本库恢复；但从未被添加进版本库的文件删掉后无法恢复。

#13 git rm --cached <file>

作用 只把文件从 Git 的跟踪列表（暂存区）中移除，本地文件保留，常用于误提交后取消跟踪。

```
$ git rm --cached readme.txt
$ git commit -m "stop tracking readme.txt"
```

注意 加了 --cached 才是“只删索引、不删文件”；漏写就会连本地文件一起删掉。取消跟踪后该文件会变成 Untracked，通常配合 .gitignore 使用。

03

第三章

远程仓库

Git 是分布式版本控制系统，同一个仓库可以分布到多台机器上，彼此没有主次。本章按“配钥匙 → 关联 → 推送 → 克隆/拉取”的顺序走完一次完整的远程协作。

3.1 配置 SSH Key

#01 **ssh-keygen -t rsa -C "youremail@example.com"**

作用 在用户主目录下生成一对 SSH 密钥：私钥 id_rsa 与公钥 id_rsa.pub。

```
$ ssh-keygen -t rsa -C "youremail@example.com"
```

注意 邮箱换成自己的，一路回车用默认值即可（无需设置密码）。生成后把 id_rsa.pub 的内容粘贴到 GitHub 的 SSH Keys 页面；私钥 id_rsa 绝不能泄露。已有 .ssh 目录和这两个文件时可直接跳到下一步。

#02 **ssh -T git@github.com**

作用 测试本机与 GitHub 的 SSH 连接是否配置成功。

```
$ ssh -T git@github.com
Hi michaelliao! You've successfully authenticated, but GitHub does not provide shell access.
```

注意 首次连接会提示 The authenticity of host ... can't be established，输入 yes 回车即可，之后不再询问。返回 Hi <用户名>! 说明公钥已生效；若报 Permission denied (publickey) 多为公钥没粘贴对或未添加到账号。

3.2 关联远程库

#03 **git remote add origin <url>**

作用 把本地仓库与一个远程仓库关联起来，并给它起名 origin。

```
$ git remote add origin git@github.com:michaelliao/learngit.git
```

注意 必须把 michaelliao 换成你自己的账户名，否则推送会因公钥不在对方账户而失败。origin 是 Git 的默认叫法，关联时必须给远程库指定一个名字。

#04 **git remote -v**

作用 查看已关联远程库的名称与 fetch/push 地址。

```
$ git remote -v
origin  git@github.com:michaelliao/learn-git.git (fetch)
origin  git@github.com:michaelliao/learn-git.git (push)
```

注意 只敲 git remote 只显示名字（如 origin）。看不到 push 地址说明你没有推送权限。地址写错时用 git remote rm origin 解除绑定后重新 add，注意这只是解除关联，远程库本身不会被删除。

3.3 推送到远程

#05 **git push -u origin master**

作用 第一次推送 master 分支的全部内容，并用 -u 把本地 master 与远程 master 关联起来。

```
$ git push -u origin master
To github.com:michaelliao/learngit.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

注意 -u 即 --set-upstream，只需第一次使用；之后可直接 git push。若远程库非空（例如建库时勾了 README），需先 git pull 再推送。

#06 **git push origin master**

作用 把本地 master 分支的最新提交推送到远程 origin。

```
$ git push origin master
To github.com:michaelliao/learngit.git
   f005ed4..7e61ed4  master -> master
```

注意 推送的是“提交”，所以本地要先 commit。若提示 ! [rejected] ... (non-fast-forward)，说明远程有你没有的提交，必须先 git pull 合并再推。

3.4 克隆与拉取

#07 **git clone <url>**

作用 把远程仓库完整复制到本地，自动建好目录、远程名 origin 与跟踪关系。

```
$ git clone git@github.com:michaelliao/gitskills.git
Cloning into 'gitskills'...
Receiving objects: 100% (3/3), done.
```

注意 也可用 https 地址（如 https://github.com/.../gitskills.git）。Git 支持多种协议，ssh 速度最快；https 每次推送都要输口令，但只开放 http 端口的公司内网只能用 https。

#08 git pull

作用 从远程抓取最新提交并合并到当前分支，是推送失败后的标准补救动作。

```
$ git pull
Auto-merging env.txt
CONFLICT (add/add): Merge conflict in env.txt
Automatic merge failed; fix conflicts and then commit the result.
```

注意 提示 There is no tracking information 说明本地分支没和远程分支建立链接，先执行 git branch --set-upstream-to=origin/dev dev。pull 产生冲突时按提示手动改文件、add、commit，再 push。

04

第四章

分支管理

分支只是指向提交的指针，创建、切换、删除都在瞬间完成。它让你可以安全地试错：在自己的分支上随便提交，完成后再合并回主线。本章从单人分支操作讲到多人协作与历史整理。

4.1 查看、创建与删除分支

#01 `git branch`

作用 列出所有本地分支，当前分支前面标一个 * 号。

```
$ git branch
* dev
  master
```

注意 只列本地分支；要看远程分支用 git branch -r，全部用 git branch -a。

#02 `git branch <name>`

作用 创建一个新分支，指针指向当前提交，但仍停留在原分支。

```
$ git branch dev
$ git checkout dev
Switched to branch 'dev'
```

注意 创建后不会自动切换，必须再 checkout。Git 新建分支只是加一个指针、改一下 HEAD，工作区文件毫无变化，所以极快。

#03 `git branch -d <name>`

作用 删除一个已经合并过的分支。

```
$ git branch -d dev
Deleted branch dev (was b17d20e).
```

注意 只能删已合并的分支，未合并时 Git 会拒绝，防止丢失修改。不能删除当前所在的分支。

#04 **git branch -D <name>**

作用 强行删除一个尚未合并的分支，丢弃它上面的全部提交。

```
$ git branch -D feature-vulcan
Deleted branch feature-vulcan (was 287773e).
```

注意 大写 D 是 --delete --force。用 -d 时会报 error: The branch 'feature-vulcan' is not fully merged，确认不要这些修改后才用 -D，删掉后提交只能靠 reflog 找回。

4.2 切换分支

#05 **git checkout <name> (git switch <name>)**

作用 切换到指定分支，HEAD 改指向该分支，工作区文件随之更新。

```
$ git checkout master
Switched to branch 'master'
```

注意 切换后原分支上未合并的提交“看不见了”，因为 master 的提交点没变。同一命令 git checkout 还负责撤销文件修改，容易混淆，所以新版 Git 提供了 git switch <name>。

#06 **git checkout -b <name> (git switch -c <name>)**

作用 创建并立刻切换到新分支，等于 git branch <name> 加 git checkout <name> 两条命令。

```
$ git checkout -b dev
Switched to a new branch 'dev'

# 新版写法
$ git switch -c dev
```

注意 -b 是 branch 的缩写，漏写就只是切换（分支不存在会报错）。从远程分支建本地分支用 git checkout -b dev origin/dev，本地与远程分支名最好一致。

4.3 合并分支

#07 git merge <name>

作用 把指定分支合并到当前分支，合并前要先切换到接收方分支。

```
$ git checkout master
$ git merge dev
Updating d46f35e..b17d20e
Fast-forward
 readme.txt | 1 +
 1 file changed, 1 insertion(+)
```

注意 Fast-forward 表示“快进合并”，只是把 master 指针前移，速度极快，但历史上看不出做过合并。若两个分支都有新提交，会报 CONFLICT，需手动编辑 <<<<<<< HEAD、=======、>>>>>>> 标记处，再 add + commit；中途放弃用 git merge --abort。

#08 git merge --no-ff -m "msg" <name>

作用 禁用快进模式合并，强制生成一个新的合并提交，让历史保留分支信息。

```
$ git merge --no-ff -m "merge with no-ff" dev
Merge made by the 'recursive' strategy.
 readme.txt | 1 +
 1 file changed, 1 insertion(+)
```

注意 因为要新建 commit，必须用 -m 写说明。团队规范通常要求合并 feature/bug 分支时加 --no-ff，这样日后能看出哪些提交来自哪次合并。

#09 git log --graph --pretty=oneline --abbrev-commit

作用 以字符图形显示分支的合并拓扑，是查看分叉与合并最直观的方式。

```
$ git log --graph --pretty=oneline --abbrev-commit
*   cf810e4 (HEAD -> master) conflict fixed
|\
| * 14096d0 (feature1) AND simple
* | 5dc6824 & simple
|/
* b17d20e branch test
```

注意 三个参数各管一件事：--graph 画图，--pretty=oneline 一行一个提交，--abbrev-commit 只显示短 id。看 rebase 前后对比时尤其有用。

4.4 储藏工作现场（stash）

#10 git stash

作用 把当前未提交的修改（工作区与暂存区）“储藏”起来，让工作区恢复干净，以便临时切换分支救急。

```
$ git stash
Saved working directory and index state WIP on dev: f52c633 add merge
```

注意 典型场景：dev 上活干到一半，却要立刻去 master 修 bug。注意未被 Git 跟踪的文件不会被 stash（除非加 -u），它们会一直留在工作区。

#11 git stash list

作用 列出所有储藏的工作现场，编号形如 stash@{0}。

```
$ git stash list
stash@{0}: WIP on dev: f52c633 add merge
```

注意 可以多次 stash，恢复时先 list 看清编号，再指定恢复某一个，如 git stash apply stash@{0}。

#12 git stash apply / git stash pop

作用 apply 恢复现场但保留储藏记录；pop 恢复的同时删掉该条记录。

```
$ git stash pop
On branch dev
Changes to be committed:
        modified:   readme.txt
Dropped refs/stash@{0} (5d677e2ee266f39ea296182fb2354265b91b3b2a)
```

注意 用 apply 恢复后必须手动 git stash drop 清理，否则 list 里会越积越多；pop 更省事。恢复后若与原文件冲突，需按冲突流程解决。

4.5 多人协作

#13 git push origin <branch>

作用 把本地某个分支推送到远程同名分支，供团队其他人拉取。

```
$ git push origin dev
To github.com:michaelliao/learngit.git
   f52c633..7a5e5dd  dev -> dev
```

注意 本地新建的分支不推送，对其他人就是不可见的。master、dev 需要与远程同步；bug 分支通常只留在本地；feature 分支是否推送取决于是否协作。推送被 rejected 时先 pull。

#14 **git branch --set-upstream-to=origin/<branch>**

作用 建立本地分支与远程分支的跟踪关系，之后 git pull / git push 可省略远程名和分支名。

```
$ git branch --set-upstream-to=origin/dev dev
Branch 'dev' set up to track remote branch 'dev' from 'origin'.
```

注意 git pull 报 There is no tracking information for the current branch 时就是缺这一步。旧写法 git branch --set-upstream dev origin/dev 已不推荐。

#15 **git checkout -b <branch> origin/<branch>**

作用 把远程分支取到本地，创建同名分支并切换过去，直接开始协作开发。

```
$ git checkout -b dev origin/dev
```

注意 clone 之后默认只有 master，别人推上来的 dev 必须先这样建出本地分支才能看到并修改。

4.6 整理历史

#16 **git rebase**

作用 变基：把本地尚未推送的分叉提交“挪”到远程最新提交之后，使提交历史变成一条直线。

```
$ git rebase
First, rewinding head to replay your work on top of it...
Applying: add comment
Applying: add author
```

注意 rebase 前后最终内容一致，但本地提交的父提交被改写了（commit id 会变）。因此只对“尚未推送”的提交做 rebase，已推送的提交变基会给协作者制造麻烦。

#17 **git cherry-pick <commit_id>**

作用 把某个特定提交所做的修改“复制”到当前分支，生成一个新的提交。

```
$ git cherry-pick 4c805e2
[master 1d4b803] fix bug 101
 1 file changed, 1 insertion(+), 1 deletion(-)
```

注意 新提交的 id（1d4b803）与原提交（4c805e2）不同：改动相同，但确实是两个提交。只复制该提交的改动，不会把整条分支 merge 过来。

05

第五章

标签管理

> 标签是指向某个 commit 的不可移动指针，用来给发布版本起一个容易记住的名字（如 v1.0）。创建和删除标签都是瞬间完成的。

5.1 创建与查看标签

#01 git tag <name>

作用 在当前提交（HEAD）上打一个标签，默认打在最新提交上。

```
$ git checkout master
$ git tag v1.0
$ git tag
v1.0
```

注意 打标签前先切到需要打标签的分支。忘了按时打标签时，可以补打在历史提交上：git tag v0.9 f52c633。

#02 git tag

作用 列出所有本地标签。

```
$ git tag
v0.9
v1.0
```

注意 标签不是按时间顺序列出，而是按字母排序的，别把顺序当成发布顺序。

#03 git show <tagname>

作用 查看某个标签指向的 commit 信息与差异内容。

```
$ git show v0.9
commit f52c63349bc3c1593499807e5c8e972b82c8f286 (tag: v0.9)
Author: Michael Liao <askxuefeng@gmail.com>

    add merge
```

注意 输出的 commit 行里带 (tag: v0.9)，可确认标签确实打在预期的提交上。

#04 **git tag -a <name> -m "msg"**

作用 创建带说明的标签（附注标签），-a 指定标签名，-m 指定说明文字。

```
$ git tag -a v0.1 -m "version 0.1 released" 1094adb
$ git show v0.1
tag v0.1
Tagger: Michael Liao <askxuefeng@gmail.com>

version 0.1 released
```

注意 末尾可跟 commit id 指定打在哪个提交上，省略则打在 HEAD。加 -a 后 git show 才会显示 Tagger 与说明文字；轻量标签只是 commit 的别名。

5.2 删除与推送标签

#05 **git tag -d <name>**

作用 删除本地标签。

```
$ git tag -d v0.1
Deleted tag 'v0.1' (was f15b0dd)
```

注意 创建的标签只存在本地，不会自动推送，所以打错了可以在本地安全删除。已推送到远程的标签，还要再删远程那份。

#06 **git push origin <tagname>**

作用 把某个本地标签推送到远程仓库。

```
$ git push origin v1.0
To github.com:michaelliao/learngit.git
 * [new tag]         v1.0 -> v1.0
```

注意 git push 默认不推标签，必须显式指定标签名。忘了推会导致别人 clone 下来看不到你的版本标签。

#07 **git push origin --tags**

作用 一次性推送所有尚未推送到远程的本地标签。

```
$ git push origin --tags
To github.com:michaelliao/learngit.git
 * [new tag]         v0.9 -> v0.9
```

注意 只推送远程没有的标签，已存在的不会覆盖。标签很多时用它比逐个推送省事。

#08 **git push origin :refs/tags/<tagname>**

作用 删除远程标签：推送一个“空”引用到该标签路径。

```
$ git tag -d v0.9
Deleted tag 'v0.9' (was f52c633)
$ git push origin :refs/tags/v0.9
To github.com:michaelliao/learngit.git
 - [deleted]         v0.9
```

注意 必须先删本地再删远程，顺序反了本地标签会被 pull 回来。命令本质是 push，冒号前为空表示删除该引用，别漏掉 refs/tags/ 前缀。

常用命令速查表 QUICK REFERENCE

命令	作用
`git init`	初始化仓库，生成 .git 目录
`git add <file>`	把文件修改放入暂存区
`git add .`	把当前目录所有新增/修改放入暂存区
`git commit -m "msg"`	提交暂存区内容，生成一条历史记录
`git status`	查看工作区与暂存区状态
`git diff`	查看工作区与暂存区的差异
`git diff HEAD -- <file>`	查看工作区与最新提交的差异
`git log`	查看提交历史
`git log --pretty=oneline`	一行一条显示提交历史
`git reset --hard HEAD^`	回退到上一个版本
`git reset --hard HEAD~n`	回退 n 个版本
`git reset --hard <commit_id>`	切换到指定版本
`git reflog`	查看 HEAD 移动记录，找回丢失提交
`git checkout -- <file>`	丢弃工作区修改（git restore <file>）
`git reset HEAD <file>`	把暂存区修改撤回工作区
`git rm <file>`	删除文件并记入暂存区
`git rm --cached <file>`	取消跟踪，保留本地文件
`ssh-keygen -t rsa -C "mail"`	生成 SSH 密钥对
`ssh -T git@github.com`	测试 SSH 连接
`git remote add origin <url>`	关联远程仓库并命名为 origin
`git remote -v`	查看远程仓库地址
`git push -u origin master`	首次推送并建立跟踪关系
`git push origin master`	推送 master 分支
`git clone <url>`	克隆远程仓库到本地
`git pull`	拉取远程更新并合并
`git branch`	列出本地分支

命令	作用
`git branch <name>`	创建分支（不切换）
`git branch -d <name>`	删除已合并分支
`git branch -D <name>`	强行删除未合并分支
`git checkout <name>`	切换分支（git switch <name>）
`git checkout -b <name>`	创建并切换分支（git switch -c）
`git merge <name>`	合并分支到当前分支
`git merge --no-ff -m "msg" <name>`	禁用快进合并，保留分支历史
`git log --graph --pretty=oneline --abbrev-commit`	图形化查看分支合并历史
`git stash`	储藏当前工作现场
`git stash list`	查看储藏列表
`git stash apply / pop`	恢复现场（pop 同时删除记录）
`git branch --set-upstream-to=origin/<b>`	建立本地与远程分支跟踪关系
`git push origin <branch>`	推送指定分支
`git rebase`	变基，把分叉历史整理成直线
`git cherry-pick <commit_id>`	复制指定提交到当前分支
`git tag <name>`	在当前提交打标签
`git tag`	列出所有标签
`git show <tagname>`	查看标签详情
`git tag -a <name> -m "msg"`	创建带说明的附注标签
`git tag -d <name>`	删除本地标签
`git push origin <tagname>`	推送单个标签
`git push origin --tags`	推送全部未推送的标签
`git push origin :refs/tags/<tag>`	删除远程标签

共 49 条。完整语法、示例与易错点见前述五章。